MANUAL DE CONTENIDOS

Python y Analítica de Datos para Profesionales del Sector Financiero

Desarrollo teórico-práctico de cada tema del programa — Globaldesk, C.A.

Este manual desarrolla, sesión por sesión y bloque por bloque, la teoría y la práctica de cada tema contemplado en el programa del curso. Cada bloque se presenta con su fundamento conceptual (Teoría) y su aplicación guiada en IDLE sobre el dataset correspondiente (Práctica), conservando el enfoque "Excel → Python" y la estadística descriptiva diseminada a lo largo de las cuatro sesiones.

SESIÓN 1 · Fundamentos de Python orientados a datos

Objetivo de la sesión: que el participante pierda el miedo al código y entienda Python como "una hoja de cálculo programable", con primer contacto informal a la estadística.

Dataset	**Bank Customer Churn Dataset (~10.000 filas)**
Fuente	*kaggle.com/datasets/gauravtopre/bank-customer-churn-dataset*
Descripción	Datos de clientes de un banco (ABC Multistate Bank): score crediticio, geografía, edad, saldo, tarjeta de crédito, actividad y abandono.
Uso pedagógico	Dataset pequeño y curado para el primer contacto, sin abrumar por volumen.

Bloque 1 — ¿Por qué Python en un banco? (1h)

Teoría

Se presenta Python como herramienta de productividad para el sector bancario, contrastándolo con Excel: mientras Excel es potente para análisis puntuales, se satura y vuelve propenso a errores humanos (fórmulas rotas, copiar/pegar) al crecer el volumen o la recurrencia de un proceso. Python permite escribir instrucciones reproducibles y auditables. Se revisan casos reales: automatización de reportes recurrentes, detección de anomalías en transacciones, modelos de scoring crediticio y conciliaciones masivas de cuentas. Como primer acercamiento informal a la estadística, se introduce la diferencia entre media y mediana con el ejemplo de "salario promedio" vs. "salario típico" en una sucursal: un solo sueldo muy alto (el del gerente) puede "inflar" el promedio, mientras la mediana refleja mejor lo que gana un empleado típico.

Práctica

Ejercicio conceptual, sin código: se presenta a los participantes una lista de sueldos ficticios de una sucursal (incluyendo un valor atípico) y se les pide estimar mentalmente el promedio y luego identificar cuál sueldo representa mejor "lo típico" del grupo. Se cierra relacionando el ejercicio con decisiones reales de negocio (¿qué cifra usaría un gerente para negociar un ajuste salarial general?).

Bloque 2 — Entorno de trabajo: IDLE (1h)

Teoría

Se explica la diferencia entre la Shell interactiva de IDLE (donde cada línea se ejecuta y evalúa de inmediato, ideal para probar ideas rápidamente) y el editor de archivos .py (donde se escribe un script completo que se guarda y se ejecuta como una unidad con la tecla F5). Se destaca por qué, para el trabajo real de análisis, conviene trabajar en scripts guardados: permiten reproducibilidad, control de versiones, corrección de errores sin perder el trabajo previo y reutilización futura — algo que la Shell, al ser volátil, no ofrece.

Práctica

Práctica guiada en IDLE:

- Abrir la Shell de IDLE y ejecutar operaciones simples línea por línea (sumas, asignación de variables).
- Crear un archivo nuevo (Ctrl+N), escribir un script breve que calcule e imprima un resultado, guardarlo como primer_script.py.
- Ejecutar el script con F5 y observar cómo el resultado aparece en la Shell, distinguiendo este flujo del trabajo línea por línea.

Bloque 3 — Sintaxis esencial (1h)

Teoría

Se cubren los bloques de construcción básicos del lenguaje: variables y asignación; tipos de datos primitivos (enteros, decimales, texto, booleanos); estructuras de datos simples como listas (colecciones ordenadas) y diccionarios (pares clave-valor); y operadores aritméticos y de comparación. Estos elementos se practican directamente en la Shell de IDLE antes de pasar a scripts más elaborados.

Práctica

Como puente hacia la estadística descriptiva, se calcula manualmente el promedio de una lista de montos de transacciones usando sum() y len(), en lugar de una función “mágica” — el objetivo es que el participante entienda exactamente qué hace .mean() por dentro antes de que Pandas se lo entregue “gratis” más adelante:

```
montos = [120.50, 340.00, 89.90, 560.25, 210.10]
promedio = sum(montos) / len(montos)
print(promedio)
```

Bloque 4 — Estructuras de control y funciones simples (1h)

Teoría

Se introducen las estructuras de control de flujo (if / elif / else) para tomar decisiones dentro de un programa, los bucles for para repetir acciones sobre una colección de datos, y la definición de funciones simples con def, incluyendo el uso de parámetros y valores de retorno como forma de organizar y reutilizar lógica.

Práctica

Práctica: clasificar montos de transacciones en rangos de riesgo (bajo/medio/alto) a partir de un valor ingresado con input(), usando condicionales para decidir a qué categoría pertenece cada monto. Como cierre estadístico del bloque, se introduce conceptualmente la moda preguntando: de los montos evaluados durante la práctica, ¿cuál rango de riesgo resultó ser el más común?

- monto = float(input("Ingrese el monto de la transacción: "))
- if monto < 100: categoria = "bajo riesgo"
- elif monto < 500: categoria = "riesgo medio"
- else: categoria = "alto riesgo"

SESIÓN 2 · Manipulación de datos con Pandas

Objetivo de la sesión: dominar el equivalente de "tablas dinámicas y filtros" en código, formalizando la estadística descriptiva básica.

Dataset	**Bank Customer Data — versión extendida (~10.000 filas + columnas extra)**
Fuente	*kaggle.com/datasets/radheshyamkollipara/bank-customer-churn*
Descripción	Extiende el dataset de la Sesión 1 con columnas de quejas, puntos ganados y tipo de tarjeta; incluye columnas irrelevantes (RowNumber, Surname).
Uso pedagógico	Practicar limpieza real con criterio de negocio: qué columnas conservar y cuáles descartar.

Bloque 1 — Introducción a Pandas y DataFrames (1h)

Teoría

Se presenta Pandas como la librería central del curso y el DataFrame como su estructura principal: una tabla con filas y columnas etiquetadas, equivalente conceptual a una hoja de Excel pero manipulable por código. Se realiza el paso explícito de instalación de la librería con pip fuera del editor (en la línea de comandos), y se explica la carga de datos desde un archivo CSV con read_csv().

Práctica

Se ejecuta el primer df.describe() del curso — momento bisagra de la formación — explicando en detalle cada fila del resultado: count (cantidad de datos no nulos), mean (promedio), std (desviación estándar), min, los percentiles 25%, 50% y 75%, y max. A partir de aquí, la estadística deja de ser un ejercicio manual y pasa a integrarse como herramienta cotidiana de exploración.

```
import pandas as pd
df = pd.read_csv("clientes_banco.csv")
print(df.describe())
```

Bloque 2 — Exploración de datos (1h)

Teoría

Se cubren las funciones esenciales de primer contacto con cualquier dataset: head() (primeras filas), info() (tipos de columna y conteo de valores no nulos) y describe() (resumen estadístico), además de la identificación de valores nulos por columna. Se refuerza la diferencia entre media y mediana usando ejemplos con outliers reales del dataset (por ejemplo, saldos o salarios con valores extremos), y se introduce la desviación estándar como medida de "dispersión" mediante la analogía de dos sucursales con el mismo saldo promedio pero un riesgo muy distinto según qué tan dispersos estén los saldos de sus clientes.

Práctica

Práctica guiada: cargar el dataset extendido, ejecutar head(), info() y describe(); identificar columnas con valores nulos con df.isnull().sum(); comparar visualmente la media y la mediana de la columna de saldo para detectar la influencia de valores extremos.

Bloque 3 — Filtrado, ordenamiento y groupby (1h)

Teoría

Se enseña el filtrado de filas mediante condiciones booleanas (equivalente a los filtros de Excel), el ordenamiento con sort_values(), y la agrupación de datos con groupby(), que permite calcular agregaciones (suma, promedio, conteo) por categoría — el equivalente directo de una tabla dinámica.

Práctica

Práctica: calcular el total de transacciones por sucursal y el promedio de saldo por segmento de cliente. Se cierra combinando agrupación con estadística — promedio y desviación estándar por sucursal o segmento — que el instructor señala como la combinación más usada en reportes bancarios reales, al permitir comparar no solo cuánto tiene cada sucursal en promedio, sino qué tan homogénea o dispersa es esa cartera.

```
df.groupby("sucursal")["saldo"].agg(["mean", "std", "count"])
```

Bloque 4 — Limpieza de datos (1h)

Teoría

Se abordan las tareas típicas de limpieza: tratamiento de valores nulos (eliminarlos con dropna() o imputarlos con fillna()), eliminación de registros duplicados (drop_duplicates()), y corrección de formatos de columna (astype()). Como cierre estadístico, se introducen los outliers a través de quantile(), presentando la pregunta central que guía la limpieza de datos en un entorno bancario real: ¿un valor extremo es un error de captura o un cliente legítimamente atípico? Los percentiles se posicionan aquí como herramienta de detección, no solo de descripción.

Práctica

Práctica integradora del bloque: limpiar y consolidar el dataset extendido de clientes — descartar columnas irrelevantes (RowNumber, Surname), tratar nulos, eliminar duplicados y usar quantile(0.01) / quantile(0.99) para identificar posibles valores atípicos en la columna de saldo, discutiendo en grupo si cada caso detectado parece un error o un cliente real.

SESIÓN 3 · Visualización y análisis exploratorio

Objetivo de la sesión: convertir datos en insumos visuales, formalizando distribución y correlación sobre volumen alto real.

Dataset	**BankSim (594.643 filas)**
Fuente	*Kaggle — simulación de banco español*
Descripción	Simula transacciones financieras de un banco en España durante ~6 meses; 7.200 de los registros son fraudulentos, con columnas de categoría, edad, género y monto.
Uso pedagógico	Primer volumen realmente alto: obliga a usar groupby y agregaciones, ya que es imposible "mirar" el dataset a simple vista.

Bloque 1 — Repaso y aplicación sobre volumen alto (1h)

Teoría

Tras un repaso breve de lo visto en la Sesión 2, se profundiza en percentiles y cuartiles, y se introduce formalmente el concepto de distribución: ¿los montos de las transacciones se concentran en algún rango particular? ¿existe una "cola larga" de valores poco frecuentes pero muy altos? Trabajar con más de medio millón de filas obliga a abandonar la inspección visual directa y apoyarse en agregaciones y resúmenes estadísticos.

Práctica

Práctica: aplicar describe() y quantile() sobre el dataset completo de BankSim (sin usar print(df) completo, siguiendo la regla de consola del curso), identificando en qué percentil se ubican los montos considerados "altos" y comentando qué tan concentrada o dispersa resulta la distribución de montos.

```
df["monto"].quantile([0.25, 0.5, 0.75, 0.90, 0.99])
```

Bloque 2 — Visualización con Matplotlib (1h)

Teoría

Se introduce Matplotlib como la librería estándar de visualización en Python, trabajando desde IDLE con la ventana emergente nativa que abre cada gráfico y el guardado de figuras con savefig(). Se cubren los tres tipos de gráfico más usados en un contexto de negocio: barras (comparar categorías), líneas (evolución en el tiempo) e histogramas (distribución de una variable numérica). El histograma se presenta como "la estadística descriptiva hecha imagen".

Práctica

Práctica: construir un histograma de los montos de transacción de BankSim y marcar visualmente, con una línea vertical, dónde cae el percentil 90 de los montos — conectando directamente la visualización con el concepto de percentil trabajado en el bloque anterior.

```
import matplotlib.pyplot as plt
p90 = df["monto"].quantile(0.90)
plt.hist(df["monto"], bins=50)
plt.axvline(p90, color="red", linestyle="--")
plt.savefig("histograma_montos.png")
plt.show()
```

Bloque 3 — Correlaciones y detección de patrones (1h)

Teoría

Se introduce la correlación como medida de qué tan relacionadas están dos variables numéricas, calculada con df.corr(). Se explica la interpretación de los valores: cercanos a 1 indican relación positiva fuerte, cercanos a -1 relación negativa fuerte, y cercanos a 0 ausencia de relación lineal — siempre aclarando que correlación no implica causalidad.

Práctica

Práctica: calcular la correlación entre el monto de la transacción y la variable de fraude (indicador binario) dentro de BankSim, discutiendo si el resultado sugiere que los montos más altos están asociados a una mayor probabilidad de fraude, y qué tan fuerte es esa asociación según el valor obtenido.

```
df[["monto", "fraude"]].corr()
```

Bloque 4 — Práctica integradora: mini-reporte visual (1h)

Teoría

Se cierra la sesión integrando todo lo trabajado en un mini-reporte visual compuesto por 3 a 4 gráficos, con hallazgos redactados en lenguaje de negocio (no técnico). El reporte debe incluir obligatoriamente cuatro elementos: una medida de tendencia central, una medida de dispersión, un percentil relevante y una interpretación de correlación.

Práctica

Práctica: cada participante (o grupo) construye su propio mini-reporte sobre BankSim combinando al menos un histograma, un gráfico de barras por categoría y una referencia a la correlación monto–fraude, cerrando con 3 a 4 frases que resuman los hallazgos como si se presentaran a un gerente de riesgo.

SESIÓN 4 · Proyecto aplicado y automatización básica

Objetivo de la sesión: consolidar todo lo aprendido en un caso bancario cercano, usando la estadística como base de una decisión de negocio.

Dataset	**Credit Card Fraud Detection — ULB Machine Learning Group (284.807 filas)**
Fuente	*Kaggle — dataset académico de referencia en detección de fraude*
Descripción	284.807 transacciones de tarjetahabientes europeos (septiembre 2013); solo 492 (0,173%) son fraudulentas. 28 de 31 columnas están anonimizadas mediante PCA (V1–V28).
Uso pedagógico	"Examen final" del curso: volumen que ya no cabe en pantalla y caso de negocio bancario clarísimo — la detección de fraude como "aguja en el pajar". Se usa para volumen y patrones (groupby por Class), no para interpretar columnas de negocio.

Bloque 1 — Exportación de resultados (1h)

Teoría

Se introduce openpyxl como librería para generar reportes automáticos en Excel directamente desde Python, incluyendo tanto los datos como gráficos exportables. Se enfatiza que un entregable profesional no es solo la tabla de datos, sino que debe incluir también un resumen estadístico (describe()) que le dé contexto a quien lo recibe.

Práctica

Práctica: exportar un subconjunto filtrado del dataset de fraude a un archivo Excel, incluyendo en una hoja adicional el resumen estadístico generado con describe().

```
with pd.ExcelWriter("reporte_fraude.xlsx") as writer:
    df.to_excel(writer, sheet_name="Datos", index=False)
    df.describe().to_excel(writer, sheet_name="Resumen")
```

Bloque 2 — Automatización de procesos repetitivos (1h)

Teoría

Se ofrece un panorama conceptual (sin profundizar en código complejo) de cómo los cálculos y reportes trabajados durante el curso pueden automatizarse para procesos recurrentes del banco: conciliaciones diarias, consolidación de reportes mensuales, y comparaciones estadísticas de un mes contra otro, ejecutadas de forma programada en lugar de manual.

Práctica

Práctica/discusión guiada: a partir de un script ya construido en el curso (por ejemplo, el de exportación a Excel del Bloque 1), identificar en grupo qué pasos serían necesarios para que ese mismo script se ejecutara automáticamente cada fin de mes con el archivo de datos actualizado.

Bloque 3 — Proyecto final guiado (1.5h)

Teoría

Se diseña, paso a paso, una mini-herramienta que lee el dataset de fraude, filtra las transacciones según un umbral de monto y/o categoría, y exporta a Excel únicamente las transacciones marcadas como sospechosas. La justificación del umbral de alerta se construye con estadística descriptiva: se comparan las medias, desviaciones estándar y

percentiles de las transacciones fraudulentas frente a las no fraudulentas, de modo que el umbral no se elige arbitrariamente sino con base en el comportamiento real de los datos.

Práctica

Práctica (proyecto guiado): agrupar el dataset por la columna Class (fraude / no fraude) y comparar sus estadísticas descriptivas; definir un umbral de monto informado por esa comparación; filtrar las transacciones que superan el umbral y exportarlas a un archivo Excel como “listing de transacciones sospechosas”.

```
df.groupby("Class")["Amount"].agg(["mean", "std", "median"])
sospechosas = df[df["Amount"] > umbral]
sospechosas.to_excel("transacciones_sospechosas.xlsx", index=False)
```

Bloque 4 — Presentación de resultados y cierre (0.5h)

Teoría

Cierre reflexivo del curso: de 284.807 transacciones, menos del 0,2% corresponden a fraude — una proporción que ilustra de forma contundente por qué el análisis de datos importa en banca: sin herramientas como las trabajadas durante el curso, encontrar esa “aguja en el pajar” de forma manual sería prácticamente inviable.

Práctica

Cada participante presenta brevemente su mini-herramienta y los hallazgos de su proyecto final. Se realiza retroalimentación grupal y se comparten recursos para continuar practicando de forma autónoma después del curso.

Referencias bibliográficas

- McKinney, W. (2022). Python for Data Analysis (3ª ed.). O'Reilly Media. Referencia estándar de la industria para análisis de datos con Python; escrita por el creador de Pandas.
- Hecho histórico: Pandas fue creado en 2008 por Wes McKinney mientras trabajaba en AQR Capital Management, un fondo de inversión cuantitativo — nació para resolver problemas de análisis financiero y series de tiempo.
- VanderPlas, J. (2016). Python Data Science Handbook. O'Reilly Media. Referencia usada para justificar la secuencia “fundamentos → Pandas → visualización → proyecto aplicado”.